

Multiple Queues

Priority classes with different time quanta

Definition

Multiple-queue scheduling uses several ready queues, usually ordered by priority. Each queue may have its own time quantum and scheduling rule.

1. Why Multiple Queues Were Introduced

Main Problem

Round robin gives every process the same quantum. This is simple, but it does not handle all workloads well.

Different Needs

Interactive processes need quick response, so they benefit from short quanta and high priority. CPU-bound processes can run efficiently with larger quanta, but they should not delay interactive work.

2. CTSS Motivation

Historical Context

One early priority scheduler was used in CTSS, the M.I.T. Compatible Time-Sharing System, on the IBM 7094.

Slow Process Switching

The IBM 7094 could hold only one process in memory. Switching processes meant writing the current process to disk and reading another one from disk. This made process switching very expensive.

Scheduling Goal

CTSS needed good interactive response while avoiding too many expensive swaps for CPU-bound processes.

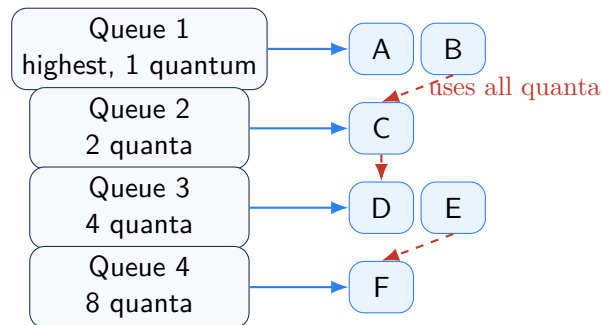
3. Small vs. Large Quantum

Quantum Choice	Benefit	Cost
Small quantum	Good response for interactive processes.	Many process switches and swaps.
Large quantum	Fewer switches for CPU-bound processes.	Poor response if everyone gets a large quantum.
Multiple queues	Combines both ideas.	More complex policy.

4. CTSS Solution

Priority Classes

CTSS used priority classes. High-priority classes ran sooner with small quanta. Lower-priority classes ran less often but received larger quanta.



Demotion Rule

Whenever a process used up all the quanta allocated to it, it was moved down one priority class.

5. Scheduling Rule

How the Scheduler Chooses

The scheduler always starts with the highest-priority nonempty queue. Processes inside the chosen queue are run round-robin style.

```
choose the highest-priority nonempty queue
run one process from that queue

if the process uses all quanta assigned to that queue then
    move it down one queue
else if the process blocks or becomes interactive then
    keep it high enough for good response
end if
```

6. Why CPU-Bound Processes Move Down

CPU-Bound Behavior

A CPU-bound process tends to use its entire allocation. Under the CTSS policy, that behavior makes it sink to lower-priority queues.

Effect

As a CPU-bound process moves lower, it runs less frequently, but when it does run, it gets a larger block of CPU time. This reduces swapping overhead.

7. 100-Quantum Example

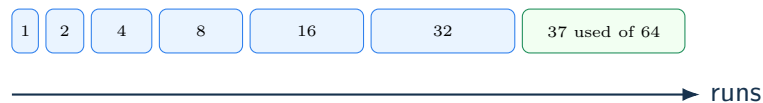
Continuous Computation

Suppose a process needs 100 quanta of CPU time without much interaction.

Run	Allocation	Total Used After Run	Result
1	1 quantum	1	Move down.
2	2 quanta	3	Move down.
3	4 quanta	7	Move down.
4	8 quanta	15	Move down.
5	16 quanta	31	Move down.
6	32 quanta	63	Move down.
7	64 quanta	100	Finishes after using only 37 of the 64.

Result

Only 7 loads or swaps are needed, including the initial load, instead of about 100 with pure round robin using one quantum at a time.



8. Why Interactive Processes Benefit

Interactive Behavior

Interactive processes often use a little CPU time and then block waiting for user input or I/O. They usually do not consume a long CPU allocation.

Good Response

Because interactive processes do not repeatedly use full quanta, they tend to remain near high-priority queues and get quick service.

9. Boosting Interactive Processes

CTSS Terminal Rule

To avoid punishing a process forever, CTSS moved a terminal's process to the highest-priority class whenever the user typed a carriage return, or Enter key.

Reason

The system assumed that pressing Enter meant the process was about to become interactive and should receive quick service.

10. Policy Can Be Gamed

Practical Problem

Users discovered that repeatedly pressing Enter could keep a CPU-bound process in a high-priority class, improving its response time unfairly.

Lesson

The policy worked in principle, but users could exploit it. Designing a scheduler that behaves well in practice is harder than designing one that sounds good in theory.

11. Multiple Queues vs. Earlier Algorithms

Algorithm	Main Rule	Main Limitation
Round robin	Every process gets the same quantum in turn.	Same quantum may be poor for mixed workloads.
Priority scheduling	Highest-priority runnable process runs first.	Low-priority processes can starve.
Multiple queues	Use priority queues with different quanta.	Policy is harder to tune and can be gamed.

12. Modern Name

Multilevel Feedback Queue

This idea is commonly known as a **multilevel feedback queue**: processes can move between priority queues based on their behavior.

Feedback

The scheduler uses feedback from past behavior. A process that uses all its CPU time looks CPU-bound and moves down. A process that behaves interactively can stay high or be boosted.

13. Big Picture

Main Conclusion

Multiple queues try to combine good interactive response with efficient handling of CPU-bound jobs. High-priority queues give short, quick turns. Lower-priority queues give larger quanta less often, reducing expensive switching.

Exam Shortcut

Remember: CTSS used multiple priority queues; lower queues got larger quanta; using all allocated quanta caused demotion; interactive activity could boost a process; bad boost rules can be exploited.

14. Quick Review

Key Points

- Multiple queues divide runnable processes into priority classes.
- CTSS used this idea because process switching required expensive swapping.
- High-priority queues used small quanta for quick response.
- Lower-priority queues used larger quanta to reduce switching overhead.
- A process that used all its allocated quanta was moved down one class.
- A CPU-bound process sinks to lower queues and runs less often but for longer.
- A 100-quantum CPU-bound job could finish with 7 swaps instead of about 100.
- Interactive processes tend to stay near higher-priority queues.
- CTSS boosted a terminal process when the user pressed Enter.
- Users could exploit that boost by pressing Enter repeatedly.
- The broader idea is now called a multilevel feedback queue.